Universidad Mariano Gálvez de Guatemala
Centro Regional de Boca Del Monte

Facultad de Ingeniería en Sistemas
CURSO: SISTEMAS OPERATIVOS II
SECCIÓN: "B"

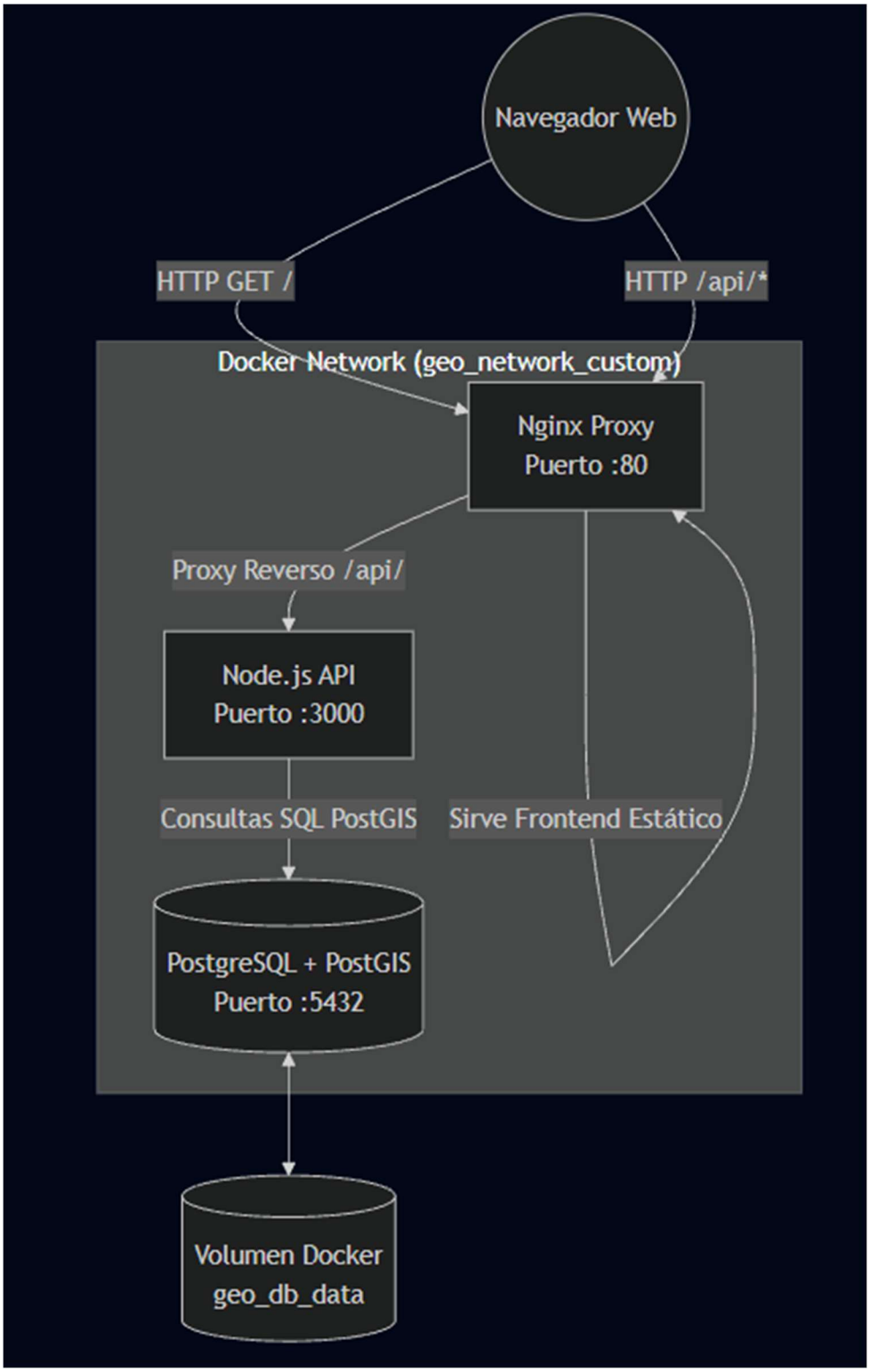
Proyecto 2

CONTENEDORES

NOMBRE: Carlos Fernando Cachin Montealegre

CARNÉ: 7690-23-17107

1. Diagrama de Arquitectura de Contenedores



2. Justificación de Tecnologías

Base de Datos (PostgreSQL + PostGIS): Elegida por ser el estándar de la industria para datos geoespaciales. PostGIS permite realizar cálculos de distancia y pertenencia geográfica directamente en el motor de base de datos mediante funciones como `ST_DWithin`, lo cual es significativamente más eficiente que procesar coordenadas en la capa de aplicación.

Servidor de Aplicaciones (Node.js + Express): Su arquitectura no bloqueante y orientada a eventos la hace ideal para manejar múltiples peticiones concurrentes de una API REST. La integración con el driver `pg` permite un manejo robusto de conexiones mediante pools, optimizando el uso de recursos.

Proxy Web y Frontend (Nginx): Utilizado como servidor de archivos estáticos para el frontend (HTML/JS/Leaflet) y como proxy inverso. Esto centraliza el acceso en un único puerto (:80) y evita problemas de seguridad como el Cross-Origin Resource Sharing (CORS) durante el desarrollo y despliegue.

3. Persistencia y Comunicación entre Servicios

Persistencia de la Información: Se garantiza mediante un volumen persistente de Docker (`geo_db_data`). Este volumen mapea la ruta interna de PostgreSQL `/var/lib/postgresql/data`, asegurando que los datos espaciales y la configuración de la base de datos se mantengan intactos incluso si los contenedores son recreados o actualizados.

Comunicación de Red (Service Discovery): El sistema utiliza una red dedicada tipo bridge (`geo_network_custom`). Dentro de esta red, los microservicios se comunican utilizando sus nombres de servicio como nombres de dominio (DNS interno). Esto permite que la API contacte a la base de datos simplemente usando el host `db`, sin depender de direcciones IP estáticas que podrían cambiar.

4. Dificultades Encontradas y Soluciones Aplicadas

Sincronización en el Arranque (Race Condition)
Problema: Inicialmente, la API intentaba conectarse a la base de datos antes de que PostgreSQL hubiera terminado de inicializarse, provocando crashes en el contenedor de Node.js.
Solución: Se implementó un `healthcheck` en el servicio de DB que utiliza `pg_isready`. Se configuró la API para depender del estado de salud de la DB (`condition: service_healthy`), garantizando que la aplicación solo inicie cuando el motor de base de datos está listo.

Visualización y Tratamiento de Coordenadas (Lon/Lat vs Lat/Lon):
Problema: Existe una confusión común entre los estándares. Mientras que las APIs de mapas (como Leaflet) suelen pedir `[Latitud, Longitud]`, las funciones espaciales de SQL esperan `Point(Longitud Latitud)`.
Solución: Se normalizó la entrada y salida de datos en la API, asegurando que el almacenamiento sea correcto mediante `ST_SetSRID(ST_MakePoint(lng, lat), 4326)` y que la visualización sea precisa.

Configuración Crítica de Variables de Nginx:
Problema: El uso de nombres de variables incorrectos en la configuración de Nginx (como `$proxy_addrs` en lugar de `$proxy_add_x_forwarded_for`) impedía el arranque del proxy reverso.
Solución: Se corrigió la configuración para adherirse a los estándares del protocolo HTTP para proxies, permitiendo la comunicación fluida entre el cliente y el backend.

Precisión en Búsquedas por Radio:
Problema: El cálculo de distancias usando el tipo `geometry` puro se realiza en grados, lo que no es útil para mediciones en el mundo real en kilómetros o metros.
Solución: Se implementó un "cast" a tipo `geography` dentro de las consultas SQL, lo que obliga a PostGIS a realizar cálculos sobre la curvatura de la tierra en metros, permitiendo que la búsqueda por radio sea exacta para el usuario final.